



DESARROLLO BIG DATA CON PYTHON

TEMA 6. Streaming, Grafos, y
PySpark en el Mundo Real



Contenidos Tema 6

- / 1 Spark Streaming: procesamiento en tiempo real**
- / 2. Manipulación de grafos**
- / 3. PySpark en el mundo real: despliegue y escalabilidad**

1. Spark Streaming: procesamiento en tiempo real

Structured Streaming

Structured Streaming es la API moderna, estándar y recomendada por Apache Spark para procesar datos en tiempo real.

- Desde Spark 2.0, el motor unifica el procesamiento por lotes (batch) que hemos visto hasta ahora, con el procesamiento en tiempo real (streaming).
- Todo se integra directamente en el módulo de SQL y DataFrames: `pyspark.sql`
- Para leer y escribir datos en streaming, usamos `spark.readStream` y `df.writeStream`

```
# --- SIMULACIÓN DE DATOS ENTRANTES EN TIEMPO REAL ---

# Lote 1: Llegan los primeros datos
with open(f"{directorio_stream}/lote1.csv", "w") as f:
    f.write("motor_principal,80\nventilador,40\n")

# Paso clave: con esta instrucción decimos a Spark que procese
# todo lo que encuentre en la carpeta donde entran datos en streaming antes de continuar.
query.processAllAvailable()
print("--- Resultados tras el Lote 1 ---")
spark.sql("SELECT * FROM tabla_resultados").show()
```

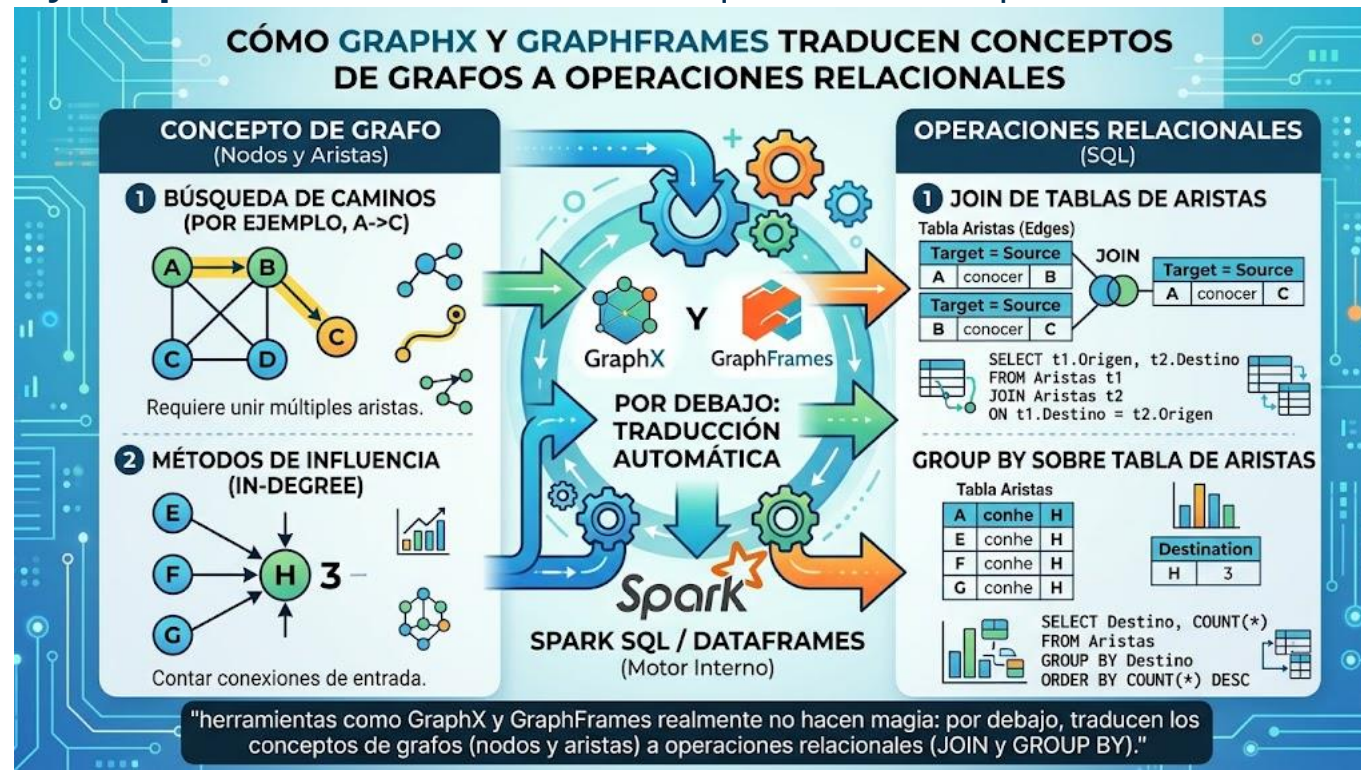
2. Manipulación de Grafos

GraphFrames

Es la librería más moderna de Spark para manejar datos en forma de grafos. Sucesora de la anterior librería GraphX.

- Se encarga de traducir conceptos de grafos (nodos y aristas) a operaciones relacionales como JOIN y GROUP BY → un grafo = dos tablas enormes de nodos y aristas

NOTA: GraphFrames y GraphX son APIs de alto nivel, que no vienen preinstaladas con PySpark por defecto.



De la libreta Colab/Jupyter al entorno de producción

- Las libretas de código (.ipynb) son ideales para explorar y dar forma a soluciones analíticas de Big Data, pero quedan muy limitadas para llevar a producción, versionar y automatizar flujos de trabajo.
- **Solución:** empaquetar el código, una vez listo para producción, en scripts Python (.py) o proyectos estructurados, y lanzar mediante el **comando** `spark-submit` o con **APIs REST**.

¿Dónde se ejecuta PySpark en el mundo real?

- Lo más frecuente es hacerlo en soluciones dedicadas de **plataformas cloud** como Amazon EMR (en AWS), Google Cloud Dataproc (en GCP) o Azure HDInsight.
- Estas soluciones levantan clústeres de procesadores bajo demanda y se apagan al terminar (pago por cuota de uso).

Alternativas:

- Plataformas gestionadas: **Databricks**, para integrar libretas, orquestación y Spark en un mismo lugar.
- Spark sobre contenedores **Kubernetes**, para arquitecturas basadas en microservicios.

¿Cómo se ejecuta? ¿Quién aprieta el botón?

- En entornos reales, los procesos (pipelines) se configuran para ejecutarse de madrugada (modo batch) o cuando llegan nuevos datos (modo streaming).

Herramientas útiles:

- **Apache Airflow:** puede llamar a PySpark y gestionar reintentos en caso de fallos.
- Alternativas a Airflow: Prefect, Dragster.

El arte de la Escalabilidad: rendimiento y anti-patrones

Cuando pasamos de 1GB a 10TB de datos, uno de los errores más comunes que hacen que un cluster de Spark colapse es *Out of Memory*. Consejos para reducir el riesgo:

- El **"shuffle"** (operaciones como *groupBy* y *join*) se convierte en el enemigo a evitar. Unir una tabla gigante con una pequeña (p.ej. Una lista de países) es más eficaz con `broadcast(tabla_pequena)`. Esto replica la tabla pequeña a todos los nodos donde se distribuye la grande, y evita un posible colapso.
- Usar siempre que sea posible **funciones nativas de Spark** (`pyspark.sql.functions`) en vez de funciones definidas por nosotros, ya que estas últimas cuando se pasan a Spark se serializan a JVM y producen una caída de rendimiento.
- Si en un script se va a usar un mismo DataFrame varias veces, usar `df.cache()` o `df.persist()`, y liberarlo al final con `df.unpersist()`.

Llevar Streaming, Grafos y ML al siguiente nivel

Streaming estructurado y robusto

- En producción, una fuente de datos en streaming puede "caerse".
- Utilizar `option("checkpointLocation", "/ruta/segura/")` para que, en caso de caída del servidor, Spark lea de este directorio al reiniciar y sepa por dónde se quedó.
- En muchas aplicaciones reales, no leeremos de ficheros CSV sino que Spark Streaming se conecta a un bus de eventos como **Apache Kafka** o **AWS Kinesis**.

MLLib y MLOps

- Entrenar un modelo de clasificación, regresión, etc., es el 10% del trabajo. Una vez llevado a producción es necesaria la monitorización del modelo.
- Mlflow juega un papel clave para ello: es gratuita y se integra nativamente con PySpark, para versionar tus modelos entrenados, las métricas utilizadas para medir su rendimiento, y servir los modelos como APIs invocables.

Procesamiento de grafos masivos

- Los algoritmos de análisis de grafos consumen muchísima memoria de forma exponencial.
- Limpiar y filtrar de forma agresiva tablas de vértices y aristas antes de crear un GraphFrame elimina nodos huérfanos o relaciones sin relevancia en nuestro análisis, evitando así desbordamientos de RAM.

PySpark como el "motor" central de procesamiento Big Data en arquitecturas modernas

- En entornos reales, es frecuente encontrarse formatos columnares como Parquet en lugar de CSVs.
- **Data Lakehouse:** unión de la flexibilidad de Data Lakes en la nube (como AWS S3) y la fiabilidad de Data Warehouses (bases de datos tradicionales).
- **PySpark** interactúa con la llamada "capa transaccional", permitiendo realizar operaciones complejas sobre grandes datos: UPDATE, DELETE, MERGE masivos, etc → prepara el terreno para Business Intelligence y Machine Learning.

